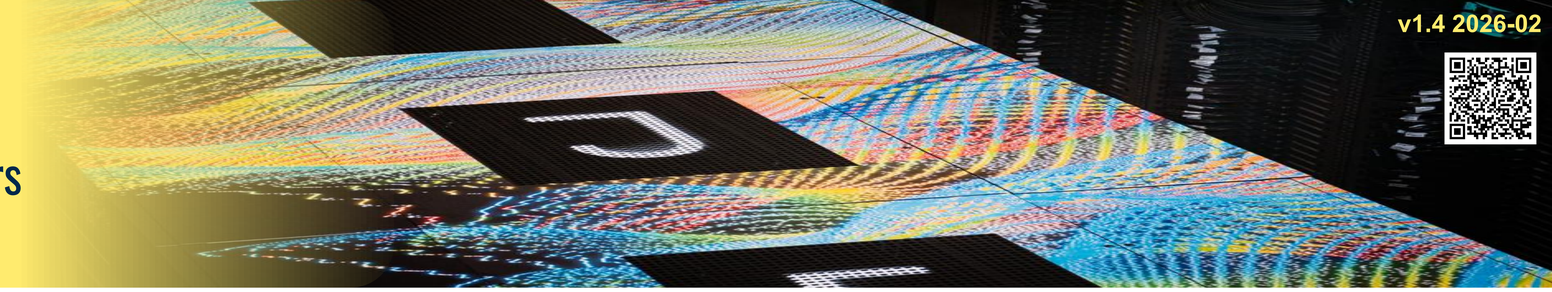
IDRIS

Democratizing LLM Fine-Tuning
Scalable SFT Workflows Across IDRIS Computing Clusters

B. Cabot & L. Hunout
N. Cassereau, M. Song, M. Comminges, K. Guerda



v1.4 2026-02

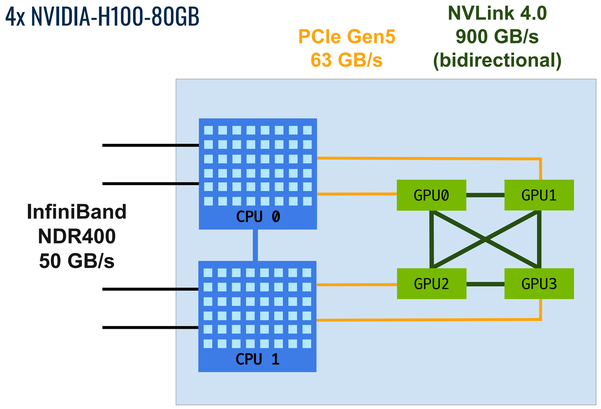
GENCI GPU Clusters at IDRIS

GENCI, the French national HPC agency, provides access to **several generations of advanced GPU systems** hosted at IDRIS. These machines support both academic and industrial research and offer the computational capacity required for modern AI workloads, including Large Language Model (LLM) training and fine-tuning.

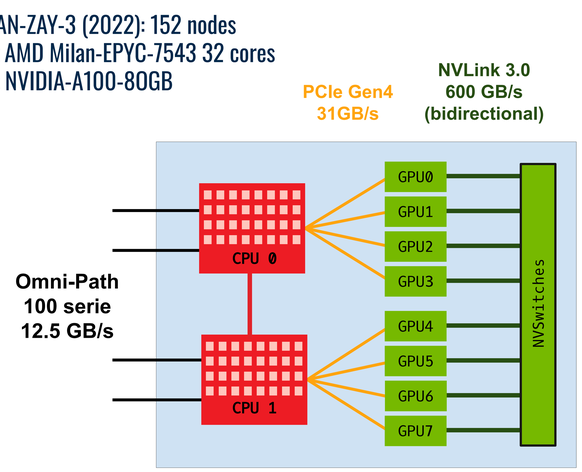
This multi-generational hardwares allows us to benchmark various LLM pre-training and fine-tuning strategies and compare their performance across systems. Our benchmarks highlight the key hardware factors that shape supervised fine-tuning performance: **GPU compute**, **memory capacity**, and **interconnect bandwidth**.

Based on these results, we provide practical reference points and resource estimates to help users plan and size their own LLM projects on IDRIS clusters. Our goal is to make such experiments more **accessible, predictable** and **transparent** for the research community.

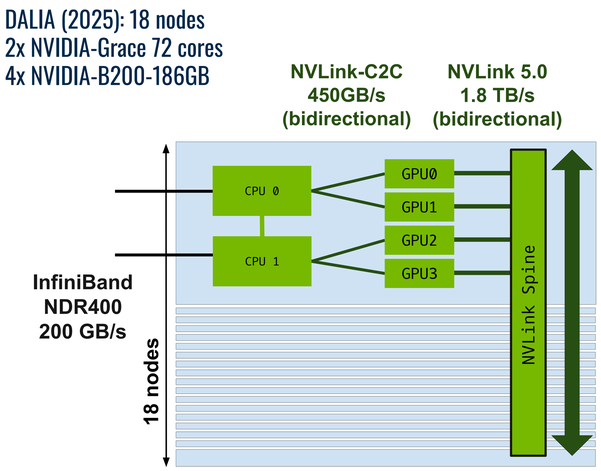
JEAN-ZAY-4 (2024): 364 nodes
2x Intel Xeon-Platinum-8468 48 cores
4x NVIDIA-H100-80GB



JEAN-ZAY-3 (2022): 152 nodes
2x AMD Milan-EPYC-7543 32 cores
8x NVIDIA-A100-80GB



DALIA (2025): 18 nodes
2x NVIDIA-Grace 72 cores
4x NVIDIA-B200-186GB



Memory as the Main Constraint in Training Strategies

LLM training is constrained primarily by two sources of GPU memory usage:

1. Activations.

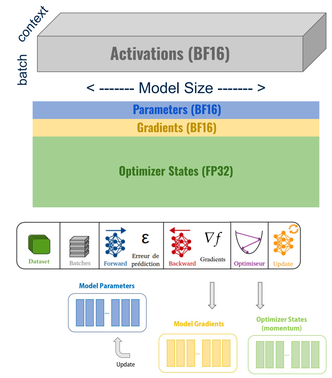
Activation memory needed for gradient computation, scales with model depth, hidden size, batch size, and sequence length. Storing all intermediate activations rapidly becomes infeasible. **Selective activation checkpointing** mitigates this by storing only the *inputs* of each Transformer block and recomputing the remaining activations during the backward pass, greatly reducing memory requirements.

2. Model states (parameters, gradients, optimizer states).

Under standard DDP, these states are fully replicated on every GPU, making large-scale LLMs prone to out-of-memory failures. As part of our multi-GPU strategy, **FSDP** shards these states across devices, making large-scale training feasible and significantly lowering the per-GPU memory footprint.

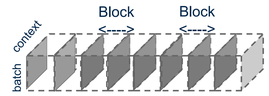
Native PyTorch Scaling Techniques

Memory Usage per GPU

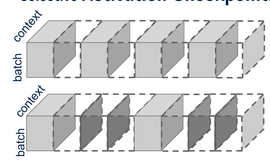


Full Activation Checkpointing

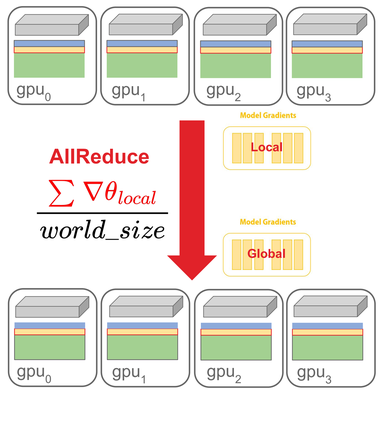
Only the **Transformer block inputs** are kept in memory; all other activations are recomputed during backpropagation.



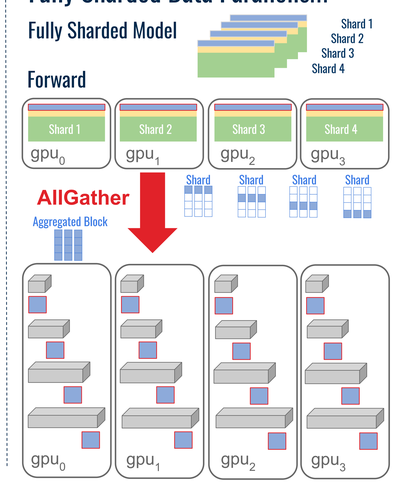
Selective Activation Checkpointing



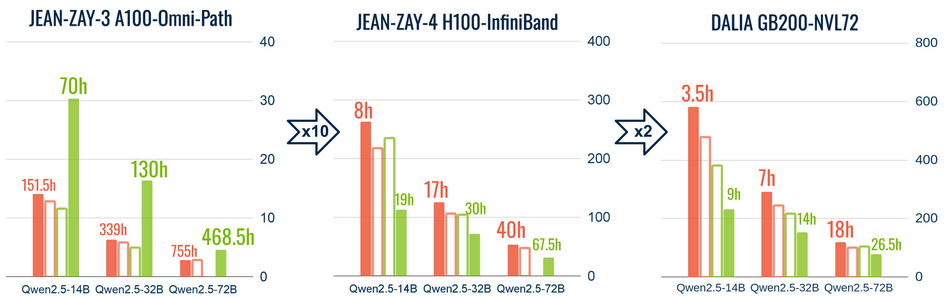
Distributed Data Parallelism



Fully Sharded Data Parallelism



SFT Benchmarking



Throughput - thousand tokens/s above bar is the overall training time

Fine-Tuning Card

Framework:
nvcr.io/nvidia/nemo:25.09
Configuration:
- A100: 8x8GPU
- H100: 16x4GPU
- GB200: 16x4GPU
Models:
Qwen2.5 (14B, 32B, 72B)
Dataset:
allenai/tulu-3-sft-mixture
Samples: 939343
Sequence Length: 4096
Global Batch Size: 128
Epochs: 2
Model Updates: 14676
Precision: bf16-mixed
Optimizer: AdamW

Practical Large-Scale LLM Fine-Tuning on IDRIS Clusters

We evaluate a realistic Instruct Fine-Tuning workflow using **64 GPUs**, a reasonable allocation on a machine like Jean Zay heavily shared among thousands of users, and also representative of a full system like DALIA. Models are **loaded directly from the Hugging Face Hub**. For this type of workload, heavy model-parallel frameworks (DeepSpeed, NeMo-Megatron, Nanotron) add complexity without providing clear benefits, as tensor or pipeline parallelism is not required at this scale.

Assuming GPUs have **≥80 GB of memory** and a **high-bandwidth interconnect**, a **PyTorch-native pipeline** based on **FSDP2**, **selective activation checkpointing**, and **torch.compile** is simple, portable, and fully sufficient. This workflow does **not** replace the multi-dimensional parallelism required for full pre-training, Mixture-of-Experts models, or very large sequence lengths, but it is well-suited for standard SFT workloads.

On systems with slower inter-node communication (e.g., Omni-Path 100), **NeMo's HFAutoModelForCausalLLM** shows a performance advantage thanks to its combined use of **FSDP** and **Tensor Parallelism**, while still allowing easy loading of pretrained models from the HF Hub.

Our results show that this **PyTorch-native** approach can fine-tune dense models up to **72B parameters** in approximately **3,000 H100 GPU-hours**, using realistic training settings (batch size of 128, context length of 4096). These results demonstrate that IDRIS hardware can **democratize large-scale LLM fine-tuning**, making it accessible to the broader scientific community.